
Durin's Code

Compiler Architecture Document

CS4031 — Compiler Construction
Spring 2026

Student Name	Student ID
Huzaifa Abdul Rehman	23K-0782
Abdul Moiz Hussain	23K-0553
Muhammad Abdullah Khan	23K-0607

Contents

1	Overview	2
2	Phase 1 — Lexer	2
2.1	Responsibility	2
2.2	Data Structures	2
2.3	Key Design Decisions	3
3	Phase 2 — Parser	3
3.1	Responsibility	3
3.2	Algorithm	3
3.3	AST Node Hierarchy	3
3.4	Error Recovery	4
4	Phase 3 — Semantic Analyser	4
4.1	Responsibility	4
4.2	Two-Pass Design	4
4.3	Symbol Table	5
4.4	Diagnostics	5
5	Phase 4 — TAC IR Generator	5
5.1	Responsibility	5
5.2	Instruction Format	5
5.3	Opcode Set	6
5.4	Temporary Allocation	6
5.5	Condition Lowering Example	6
6	Phase 5 — Optimiser	6
6.1	Responsibility	6
6.2	Pass 1: Dead Code Elimination	7
6.3	Pass 2: Redundant AND Folding	7
7	Phase 6 — Code Generator	7
7.1	Responsibility	7
7.2	JSON Bytecode Format	7
8	Phase 7 — Virtual Machine	8
8.1	Responsibility	8
8.2	Game State	8
8.3	Execution Model	8
8.4	Game Loop	8
9	Build System	9
10	Third-Party Dependencies	9

1 Overview

The Durin's Code compiler is organised as a linear seven-phase pipeline. Each phase has a well-defined input type and output type; phases communicate only through these typed interfaces, enabling independent development and testing.

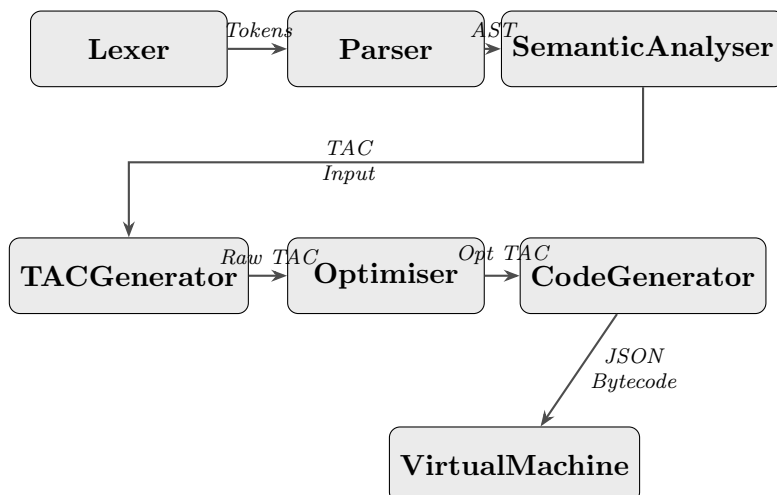


Figure 1: Durin's Code compilation pipeline

#	Phase	Input	Output
1	Lexer	Raw source <code>char*</code>	Token stream
2	Parser	Token stream	ProgramNode AST
3	Semantic Analyser	ProgramNode	SemanticResult + SymbolTable
4	TAC Generator	AST + SymbolTable	TACProgram
5	Optimiser	TACProgram	Optimised TACProgram
6	Code Generator	AST + TACProgram	JSON bytecode string
7	Virtual Machine	JSON bytecode	Interactive game session

2 Phase 1 — Lexer

2.1 Responsibility

Convert a null-terminated C-string source into `Token` structs on demand (*pull model* — `scanToken()` is called once per token, lazily).

2.2 Data Structures

```

1 struct Lexer {
2     const char* start;          // start of current lexeme
3     const char* current;       // scan head
4     int line, column;
5     TokenType lastEmitted;
6     TokenType lastBeforeDot; // for context-sensitive dot handling
7 };
8

```

```
9 struct Token {
10     TokenType    type;
11     const char*  start;    // pointer into original source (no copy)
12     int          length;
13     int          line, column;
14 };
```

Listing 1: Lexer state and Token struct

2.3 Key Design Decisions

Pull model. The parser calls `scanToken()` whenever it needs the next token. There is no buffering; the lexer reads one character at a time. This minimises memory use for large source files.

Context-sensitive dot tokens. When a `.` is scanned, `lastBeforeDot` records the previous token type. The next identifier is reclassified:

- `player.x` $\Rightarrow$ `TOKEN_PLAYER_ATTR`
- `id.x` $\Rightarrow$ `TOKEN_ROOM_ATTR`

This avoids adding semantic lookahead to the parser.

Line/column tracking. The `advance()` function is the single point that increments `lexer.line` and resets `lexer.column` on `\n`. No other function touches these counters, preventing double-counting bugs. Windows `\r\n` endings are consumed as a single newline.

Error recovery. A lexer error emits a `TOKEN_ERROR` token pointing to a static error string. Scanning stops; the parser detects `TOKEN_ERROR` and reports the failure.

3 Phase 2 — Parser

3.1 Responsibility

Consume the token stream and build a fully typed Abstract Syntax Tree rooted at `ProgramNode`.

3.2 Algorithm

Hand-written recursive descent. Each grammar production maps 1-to-1 to a `parseXxx()` function. No parser-generator is used.

3.3 AST Node Hierarchy

```
1 struct ASTNode {
2     NodeType type;
3     int line, column;
4 };
```

```

5
6 struct ProgramNode : ASTNode {
7     std::vector<std::unique_ptr<ASTNode>> declarations;
8 };
9
10 struct RoomDeclNode : ASTNode {
11     std::string name, description;
12     std::vector<std::unique_ptr<ItemDeclNode>> items;
13     std::vector<std::unique_ptr<NpcDeclNode>> npcs;
14     std::vector<std::unique_ptr<ExitDeclNode>> exits;
15 };
16
17 struct ActionDeclNode : ASTNode {
18     std::string name;
19     std::vector<std::unique_ptr<StmtNode>> body;
20 };
21
22 // Statements: PrintStmtNode, RemoveStmtNode, AssignStmtNode,
23 //              IfStmtNode
24 // Conditions: ConditionNode { type, lhs, rhs, left, right }

```

Listing 2: AST node types (simplified)

All child ownership is expressed through `unique_ptr`. There is no manual `delete` anywhere in the parser.

3.4 Error Recovery

On a parse error the parser calls `synchronize()`, which advances past statement-boundary tokens (`}`, `room`, `action`). This allows multiple parse errors to be reported in a single compilation.

4 Phase 3 — Semantic Analyser

4.1 Responsibility

Statically validate the AST against all language rules and produce a `SymbolTable` consumed by downstream phases.

4.2 Two-Pass Design

The analyser runs two passes over the AST:

Pass	Function	Action
1	<code>registerDeclarations()</code>	Walk all top-level declarations; insert rooms, items, NPCs, and actions into the <code>SymbolTable</code> . Report duplicates.
2	<code>validateDeclarations()</code>	For each room: verify exit targets exist and no direction is repeated. For each action: validate <code>remove</code> , <code>has_item</code> , and <code>current_room</code> references.

A two-pass design is necessary because exits may reference rooms declared later in the file. A single-pass approach would incorrectly report forward references as errors.

4.3 Symbol Table

```
1 struct SymbolTable {  
2     std::unordered_map<std::string, RoomInfo>    rooms;  
3     std::unordered_map<std::string, ItemInfo>    items;  
4     std::unordered_map<std::string, NpcInfo>     npcs;  
5     std::unordered_map<std::string, ActionInfo>  actions;  
6     std::unordered_map<std::string, std::string> playerAttrTypes;  
7 };
```

Listing 3: SymbolTable structure

4.4 Diagnostics

Errors and warnings are collected into a `vector<Diagnostic>` with severity, line, column, and message fields. All diagnostics are printed to `stderr`. If any `ERROR`-severity diagnostic exists, `hadError = true` and compilation halts before TAC generation.

5 Phase 4 — TAC IR Generator

5.1 Responsibility

Lower the validated AST into a flat sequence of Three-Address Code instructions — one `TACAction` per action declaration.

5.2 Instruction Format

```
1 struct TACInstruction {  
2     TACOp    op;  
3     std::string dest, src1, src2;  
4     int      intValue;  
5     bool     boolValue;  
6     std::string strValue;  
7 };
```

Listing 4: TAC instruction struct

5.3 Opcode Set

Opcode	Semantics
ASSIGN	<code>dest = src1</code>
ADD / SUB	Arithmetic into <code>dest</code>
COMPARE_EQ/GT/GTE/LT/LTE	Comparison result (bool) into <code>dest</code>
AND / OR	Logical combine of two bool temporaries
JUMP_IF_FALSE	Branch to label in <code>strValue</code> if <code>src1</code> is false
JUMP	Unconditional branch to label in <code>strValue</code>
LABEL	Branch target; name in <code>strValue</code>
PRINT	Print string <code>strValue</code>
REMOVE_ITEM	Remove item <code>src1</code> from world
SET_PLAYER_ATTR	Set player attribute <code>dest</code> from <code>src1</code> with op in <code>strValue</code>
HAS_ITEM	Check inventory for <code>src1</code> ; result into <code>dest</code>
CHECK_ROOM	Check <code>currentRoom == strValue</code> ; result into <code>dest</code>

5.4 Temporary Allocation

Temporaries are named `t0`, `t1`, `t2`, ..., allocated by a counter reset at the start of each action.

5.5 Condition Lowering Example

The compound condition `current_room == "mount_doom" && player.has_item(the_ring)` is lowered to:

```

t0  <- CHECK_ROOM    "mount_doom"
t1  <- HAS_ITEM      the_ring
t2  <- AND            t0  t1
      JUMP_IF_FALSE t2  L_else_0
      ...then body...
      JUMP          L_end_0
L_else_0:
      ...else body...
L_end_0:

```

6 Phase 5 — Optimiser

6.1 Responsibility

Apply machine-independent optimisations to reduce redundant instructions before code generation.

6.2 Pass 1: Dead Code Elimination

After an unconditional JUMP instruction, all instructions up to (but not including) the next LABEL are unreachable and are removed.

Listing 5: Dead code elimination

BEFORE:	AFTER:
JUMP L_end	JUMP L_end
PRINT "dead" <- removed	
ASSIGN t0, 1 <- removed	
LABEL L_end	LABEL L_end

6.3 Pass 2: Redundant AND Folding

When an AND instruction has both operands identical (AND t2, t0, t0), t2 is a pure alias of t0. The instruction is removed and all subsequent uses of t2 are rewritten to t0.

This situation arises from single-condition if statements that go through the general condition-combining code path.

Both passes run on each TACAction independently, so they are trivially parallelisable.

7 Phase 6 — Code Generator

7.1 Responsibility

Serialise the world model and optimised TAC into a structured JSON bytecode string or file using the vendored nlohmann/json v3.11.3 library.

7.2 JSON Bytecode Format

Listing 6: JSON bytecode schema

```
{
  "world": [
    {
      "id": "bag_end",
      "description": "The cozy hole of a Hobbit...",
      "items": [ { "name": "the_ring", "props": { "power": 100 } } ],
      "npcs": [],
      "exits": [ { "direction": "east", "target": "buckland" } ]
    }
  ],
  "start_room": "bag_end",
  "actions": [
    {
      "name": "take ring",
      "instructions": [
        { "op": "CHECK_ROOM", "dest": "t0", "strValue": "bag_end" },
        { "op": "JUMP_IF_FALSE", "src1": "t0", "strValue": "L_else_0" },
        { "op": "SET_PLAYER_ATTR", "dest": "inventory",
```



```

        "src1": "the_ring",      "strValue": "+=" },
    { "op": "PRINT",              "strValue": "The Precious is yours." },
    { "op": "LABEL",              "strValue": "L_end_0" }
  ]
}
]
}

```

The output is pretty-printed with 2-space indentation. This format is the handoff point between the compiler and the VM; it is also what the `-o` flag writes to disk.

8 Phase 7 — Virtual Machine

8.1 Responsibility

Load JSON bytecode, maintain game state, and run the interactive game loop.

8.2 Game State

```

1 struct GameState {
2     std::string                currentRoom;
3     std::unordered_set<std::string> inventory;
4     std::unordered_map<std::string,int> intAttrs;
5     std::unordered_map<std::string,bool> boolAttrs;
6     std::unordered_map<std::string,std::string> strAttrs;
7 };

```

Listing 7: Runtime game state

8.3 Execution Model

`executeAction()` builds a `label → index` map from the instruction list, then runs an instruction-pointer (`ip`) loop:

1. Fetch instruction at `ip`.
2. Dispatch on `op` via `switch` statement.
3. Update `ip` (`ip++` normally; `ip = labelMap[target]` for branches).
4. Repeat until `ip` goes past the last instruction.

A `std::unordered_map<std::string, Value>` register file stores temporary values (`t0`, `t1`, ...) during execution.

8.4 Game Loop

`runGameLoop()` provides the player interaction:

```

=== Durin's Code Adventure ===
Commands: look, inventory, go <dir>, quit, or any action name.

=== bag_end ===
The cozy hole of a Hobbit. A golden ring glints on the table.
Items here: the_ring
Exits: east -> buckland

> take ring
The Precious is yours.

> go east
=== buckland ===
...

```

Listing 8: Sample game session

Built-in commands: `look`, `inventory`, `go <dir>`, `quit`. Any other input is matched against action names; unrecognised input prints a “don’t understand” message. After each action the VM checks `state.boolAttrs["win"]`; if true, the game prints the win message and exits.

9 Build System

CMake 3.23+ with one static library per phase:

```
durins_lexer ← durins_parser ← durins_semantic ← durins_tac ← durins_codegen ← durins_vm
```

Each library exposes its `src/` subtree as a public include directory. Test executables link only the library under test plus GoogleTest, keeping compilation units small and test feedback fast.

Target	Source	Links
<code>durins_lexer</code>	<code>src/lexer/lexer.cpp</code>	—
<code>durins_parser</code>	<code>src/parser/parser.cpp</code>	lexer
<code>durins_semantic</code>	<code>src/semantic/semantic.cpp</code>	parser
<code>durins_tac</code>	<code>src/tac/tac.cpp</code> , <code>optimizer.cpp</code>	semantic
<code>durins_codegen</code>	<code>src/codegen/codegen.cpp</code>	tac
<code>durins_vm</code>	<code>src/vm/vm.cpp</code>	codegen
<code>durinsc</code>	<code>src/main.cpp</code>	vm

10 Third-Party Dependencies

Library	Version	Location	Purpose
nlohmann/json	3.11.3	<code>third_party/nlohmann/json.hpp</code>	JSON serialisation
GoogleTest	system	package manager	Unit testing

`nlohmann/json` is vendored as a single header file, eliminating any network dependency at build time.